



NATURAL LANGUAGE PROCESSING

Project Title

HATE SPEECH DETECTION USING NLP

Programme	BBA (Artificial Intelligence)
Academic Year	2026-27
Team Members	1. Jashan Chaudhary - 25030422054 2. Sanjeev Katragadda - 25030422059 3. Agustyae Gupta - 25030422011 4. Ansh Sahani - 25030422016
Faculty	Dr. Nabanita Choudhury, Assistant Professor

SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE (SAII)

Constituent of Symbiosis International (Deemed University)

Lavale, Pune - 412115

1. Abstract

This project develops a Natural Language Processing (NLP) system for automatic hate-speech detection in online comments. The project uses the ETHOS Hate Speech Dataset, an English-language dataset containing 998 comments with binary hate-speech labels. The original hate-score field is converted into a binary target using a threshold of 0.5. After basic data cleaning, the dataset is split into stratified training and testing sets. Text is represented using Term Frequency–Inverse Document Frequency (TF-IDF) features with unigrams and bigrams, and a Logistic Regression classifier is trained with class balancing. The model achieves 69.5% accuracy, 67.6% hate-class precision, 57.5% hate-class recall, 62.1% hate-class F1-score and a ROC-AUC of 0.727 on the held-out test set. Beyond overall performance, the project audits two text styles: short versus long comments and comments with versus without profanity. The profanity audit shows a substantial difference in hate recall, suggesting that the model may rely on obvious offensive vocabulary as a shortcut. A simple explainability analysis based on Logistic Regression coefficients identifies the words and phrases that most strongly influence predictions. The project is intended as an educational moderation-support baseline rather than an autonomous moderation system.

2. Introduction

Online platforms receive large volumes of user-generated text, making manual review of every comment difficult. Hate speech can negatively affect users and communities, while an automated system can provide a first-pass signal for human moderators. At the same time, automatic hate-speech detection is challenging because harmful meaning may depend on context, target, sarcasm, slang, or the difference between mentioning a sensitive term and directing abuse at a person or group.

NLP provides techniques for converting natural-language text into machine-readable representations and learning patterns that support classification. In this project, TF-IDF is used to transform comments into numerical features, and Logistic Regression is used as a transparent baseline classifier. This combination is appropriate for a small text-classification dataset and also supports a simple explainability analysis because the learned feature coefficients can be inspected. Scikit-learn describes TF-IDF as a transformation from raw documents into a TF-IDF-weighted document-term matrix, while Logistic Regression provides a linear classification model whose coefficients can be examined for feature influence. [3][4]

The project was selected as a mini NLP problem because it satisfies the assessment requirement for a complete pipeline while also allowing the team to evaluate the model beyond a single overall score. The audit component examines whether performance changes across different writing styles, and the explainability component examines which textual features are driving the model.

3. Problem Statement and Objectives

The problem is to automatically classify short online comments as hate speech or non-hate speech using NLP and a supervised machine-learning approach.

Objectives

- Build a working NLP classification pipeline using a real hate-speech dataset.
- Clean and prepare the text data and create a reproducible train/test split.
- Represent comments using TF-IDF features and train a Logistic Regression classifier.

- Evaluate the system using accuracy, precision, recall, F1-score, ROC-AUC and a confusion matrix.
- Audit model performance across at least two text subgroups/styles: short versus long comments and profanity versus no profanity.
- Provide a simple explainability analysis using the learned TF-IDF feature coefficients.
- Document limitations and responsible-use considerations in a one-page model card.

4. Dataset Description

The project uses the ETHOS binary hate-speech dataset. The official ETHOS repository describes the binary file as containing 998 comments, of which 433 contain hate speech and 565 do not. The dataset was created for hate-speech detection and is based on online comments. The original ETHOS work describes binary and multi-label variants and explains the annotation process. [1][2]

Item	Description
Dataset	ETHOS Hate Speech Dataset – Binary
Source	Kaggle / ETHOS project repository
Records	998 comments
Fields used	`comment`, `isHate`
Language	English
Hate class	433 comments (43.4%)
Non-hate class	565 comments (56.6%)
Label rule	`isHate >= 0.5` → Hate; otherwise Non-Hate
Split	80% train / 20% test, stratified, random_state = 42
Test set	200 comments: 87 hate, 113 non-hate

Table 1. Summary of the ETHOS binary dataset used in the project.

The dataset is moderately imbalanced toward non-hate comments. The model therefore uses `class\_weight='balanced'` during training. The notebook reports no removal of exact duplicates from the final 998 comments after the cleaning checks, and the median comment length in the test set is 14 words.

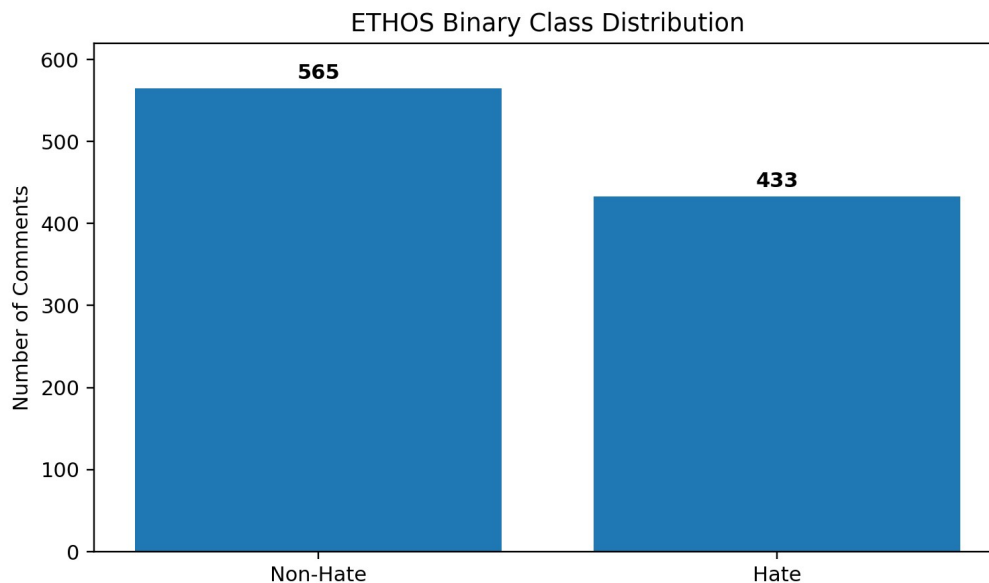


Figure 2. ETHOS class distribution in the dataset.

5. Data Preprocessing

Only preprocessing actually performed in the notebook is reported here. The dataset is reduced to the `comment` and `isHate` fields, the text is converted to string format and leading/trailing whitespace is removed. Missing text and empty comments are removed, and exact duplicate comments are dropped. The continuous `isHate` score is then converted to the binary target used for classification.

- Lowercasing is handled by the TF-IDF vectorizer (`lowercase=True`).
- Accents are normalized using `strip\_accents='unicode'`.
- No manual stop-word removal is performed.
- No stemming or lemmatization is performed.
- No separate URL, hashtag, mention or emoji-removal stage is implemented.
- No multilingual or code-mixed processing is performed; the project is limited to English text.

These choices keep the pipeline simple and reproducible. They also contribute to the model's limitations because raw lexical signals can dominate when the system has no explicit treatment for context, sarcasm, or linguistic normalization.

6. Feature Representation / Embeddings

The project uses TF-IDF (Term Frequency–Inverse Document Frequency) rather than a dense embedding model. TF-IDF represents each comment as a sparse numerical vector where terms are weighted according to their occurrence in a document relative to their frequency in the corpus. The implementation uses word-level unigrams and bigrams so the model can capture both individual words and short two-word phrases. The vectorizer is configured with `min\_df=2`, `max\_df=0.95`, `sublinear\_tf=True`, lowercase processing and Unicode accent stripping. [3]

TF-IDF was selected because the dataset is small, the feature representation is computationally lightweight, and the resulting features can be directly connected to the coefficients of a linear classifier for explainability. This makes it appropriate for a baseline academic project where interpretability is an assessment objective.

7. Proposed Methodology / System Architecture

The complete system is implemented as a single scikit-learn Pipeline. The workflow starts with the raw ETHOS comments, performs basic cleaning, converts the text into TF-IDF unigram/bigram features, and passes those features to Logistic Regression. The classifier returns a binary prediction of Hate or Non-Hate, while probability scores are retained for audit analysis.

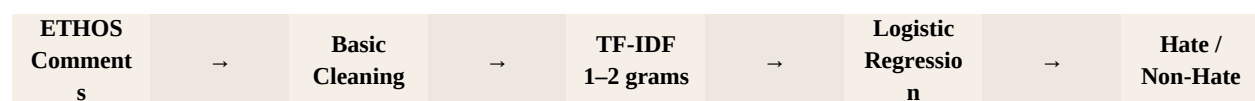


Figure 1. End-to-end workflow of the hate-speech detection pipeline.

For model auditing, the held-out test set is further divided into short/long groups based on the median word count and profanity/no-profanity groups using a fixed list of 11 profanity terms. These slices are evaluated separately using accuracy, precision, recall and F1-score.

8. NLP Model / Technique Used

Logistic Regression

Logistic Regression is used as the supervised classifier. In the binary setting, the model learns weights over the TF-IDF features and converts the resulting linear score into a probability for the positive class. The scikit-learn implementation supports sparse input, making it suitable for TF-IDF matrices, and exposes learned coefficients that can be inspected for interpretability. [4]

Parameter	Setting
Classifier	LogisticRegression
Maximum iterations	2000
Class balancing	class_weight = balanced
Random state	42
TF-IDF n-grams	(1, 2)
Minimum document frequency	2
Maximum document frequency	0.95
Sublinear TF	True

Table 2. Main model and TF-IDF settings used in the notebook.

The model is trained once on the training split and evaluated on the held-out test split. No cross-validation or hyperparameter search is performed, so the reported results should be treated as a baseline rather than an optimized benchmark.

9. Implementation

The project is implemented in Python using Google Colab. The main libraries are Pandas and NumPy for data handling, scikit-learn for the NLP pipeline and evaluation metrics, Matplotlib and Seaborn for visualizations, and KaggleHub for dataset download.

Component	Implementation
Language	Python
Environment	Google Colab
Data handling	Pandas, NumPy
NLP representation	scikit-learn TfidfVectorizer
Classifier	scikit-learn LogisticRegression
Evaluation	accuracy, precision, recall, F1, ROC-AUC, confusion matrix
Audit	Pandas-based subgroup slicing
Explainability	Logistic Regression coefficient inspection

Table 3. Software components used in the implementation.

No separate web or desktop user interface was developed for this assessment. The working system is the Colab notebook itself, which loads the data, trains the model, evaluates it, performs the subgroup audit and prints the explainability output. The complete implementation is intended to be maintained in the associated GitHub repository.

10. Results and Evaluation

The model is evaluated on 200 held-out comments. Because the task is binary classification and the hate class is the main target of interest, precision, recall and F1-score are reported alongside accuracy and ROC-AUC.

Metric	Overall / Hate class	Value
Accuracy	Overall	0.695 (69.5%)
Precision	Hate	0.676 (67.6%)
Recall	Hate	0.575 (57.5%)
F1-score	Hate	0.621 (62.1%)
ROC-AUC	Overall	0.727
Precision	Non-Hate	0.706 (70.6%)
Recall	Non-Hate	0.788 (78.8%)
F1-score	Non-Hate	0.745 (74.5%)
Macro F1	Both classes	0.683 (68.3%)

Table 4. Test-set performance of the TF-IDF + Logistic Regression model.

The confusion matrix contains 89 true non-hate predictions, 24 false positives, 37 false negatives and 50 true hate predictions. The model therefore identifies 50 of 87 hate comments while missing 37. Accuracy is 69.5%, which is above the 56.5% majority-class baseline but also shows that the simple baseline model has substantial room for improvement.

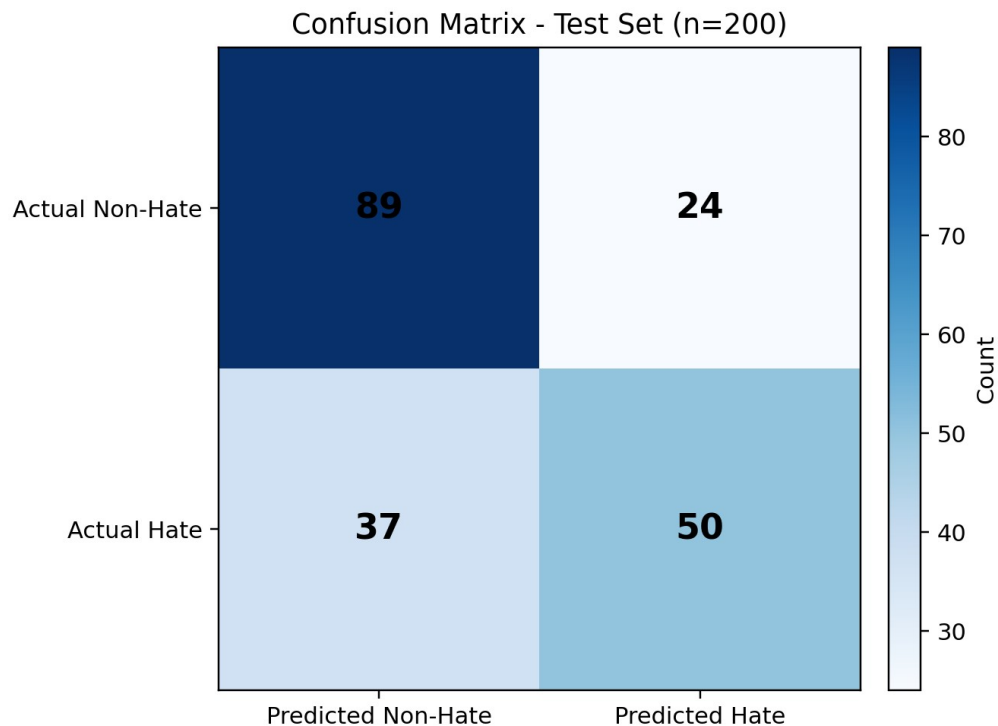


Figure 3. Confusion matrix for the held-out test set.

Actual \ Predicted	Non-Hate	Hate
Non-Hate	89	24
Hate	37	50

Table 5. Confusion matrix for the 200-comment test set.

Performance audit

Audit group	Samples	Accuracy	Hate Recall	Precision	F1
Short comments	104	68.3%	59.5%	61.0%	60.2%
Long comments	96	70.8%	55.6%	75.8%	64.1%
Profanity	31	67.7%	89.5%	68.0%	77.3%
No profanity	169	69.8%	48.5%	67.3%	56.2%

Table 6. Performance across two text subgroups/styles.

The short-versus-long audit shows relatively small changes in accuracy and recall, although precision is higher for long comments. The larger difference appears in the profanity audit: hate recall is 89.5% when profanity is present but only 48.5% when profanity is absent. In addition, 8 of 12 non-hate comments containing profanity are incorrectly flagged, indicating that profanity may act as a shortcut feature. The subgroup audit is a text-style audit, not a demographic fairness audit.

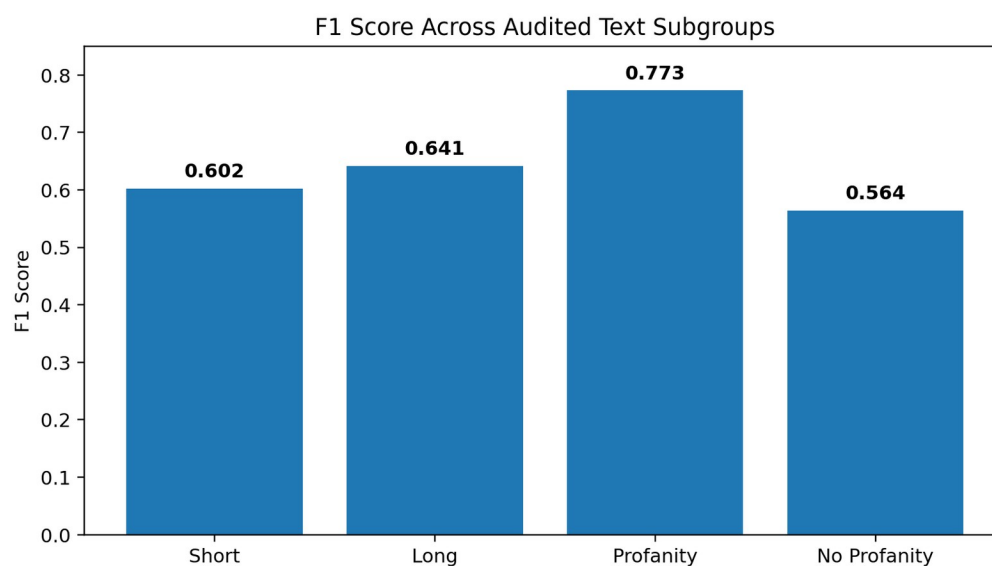


Figure 4. F1-score across the audited text subgroups.

11. Error Analysis / NLP Failure Analysis

The main failure pattern is context dependence. The model works from word and short-phrase associations rather than understanding the intent of the full sentence. As a result, explicit offensive vocabulary can increase the hate score even when the comment is not classified as hate in the dataset, while hate expressed without obvious profanity can be missed.

Representative demonstration cases

Text	Model output	Hate probability	Interpretation
"People from that group should leave"	Non-Hate	0.36	Illustrates a possible miss caused by weak lexical evidence despite exclusionary language.
"I disagree with your opinion"	Hate	0.57	Illustrates a false-positive tendency when the model overweights learned lexical patterns.

Table 7. Qualitative demonstration examples used for error analysis; these are not part of the held-out test set.

The audit also shows that profanity-containing non-hate comments are frequently false positives. This behaviour is consistent with the feature-level analysis, where strongly hate-leaning features include offensive vocabulary and identity-related terms. The model therefore has difficulty distinguishing the presence of a word from the intent or context in which that word is used.

12. Ethical Considerations and Limitations

Bias and fairness

The audit in this project focuses on text styles rather than demographic subgroups because the binary ETHOS file does not provide a direct demographic attribute for every example. The profanity audit reveals a meaningful performance gap and suggests that the model may learn a shortcut association between profanity and hate. Identity-related words can also receive strong positive coefficients, which creates a risk of false flags when groups are discussed in neutral or non-hateful contexts.

Limitations

- The dataset contains only 998 comments, and the held-out test set contains 200 comments, so subgroup estimates can be noisy.
- The model misses a substantial proportion of hate comments; hate-class recall is 57.5%.
- TF-IDF + Logistic Regression does not explicitly model context, sarcasm, negation scope or conversation history.
- The analysis is English-only and may not generalize to multilingual, code-mixed or platform-specific language.
- The binary label is derived from a 0.5 threshold on the original hate score, so edge cases may reflect annotation uncertainty.
- The project uses one train/test split and does not perform cross-validation or hyperparameter tuning.

For these reasons, the system should be used only as a first-pass moderation-support tool with human review. It should not be used as the sole basis for automatic removal, account bans or other punitive actions.

13. Conclusion

This project successfully implements a complete NLP pipeline for binary hate-speech detection using the ETHOS dataset. The pipeline performs data cleaning, stratified train/test splitting, TF-IDF feature extraction and Logistic Regression classification. The model achieves 69.5% accuracy and a hate-class F1-score of 62.1% on the held-out test set. The subgroup audit extends the evaluation beyond overall performance and identifies a clear difference between comments with and without profanity. The explainability analysis also shows that the classifier is strongly influenced by specific lexical features. Overall, the project meets its objectives and demonstrates both the usefulness and limitations of a simple, interpretable NLP baseline for hate-speech detection.

14. Future Scope

- Use character n-grams and compare alternative linear classifiers such as Linear SVM.
- Use cross-validation and a validation split for threshold selection and hyperparameter tuning.

- Expand the audit to include identity-term swaps and other controlled text-style tests.
- Evaluate transformer-based models such as BERT or RoBERTa to capture context and phrase-level semantics.
- Increase dataset size and include more recent, multilingual and platform-diverse comments.
- Calibrate prediction thresholds for a moderation setting where recall may be prioritized over raw accuracy.

15. References

1. [1] ETHOS Hate Speech Dataset, Kaggle. “ETHOS Hate Speech Dataset.” Available at: <https://www.kaggle.com/datasets/konradd/ethos-hate-speech-dataset>
2. [2] I. Mollas, Z. Chrysopoulou, S. Karlos, and G. Tsoumakas, “ETHOS: an Online Hate Speech Detection Dataset,” arXiv:2006.08328, 2020. DOI: <https://doi.org/10.1007/s40747-021-00608-2>
3. [3] Scikit-learn Documentation, “TfidfVectorizer.” Available at: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
4. [4] Scikit-learn Documentation, “LogisticRegression.” Available at: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

16. GitHub Repository

GitHub Repository Link: To be added before final submission.

The repository should contain the final Jupyter notebook, dataset or a reproducible dataset-download step, README.md, and the supporting report/model-card artifacts required for assessment.

17. Team Contribution

The contribution allocation below is a structured draft based on the project deliverables and should be confirmed by the team before final submission.

Team Member	PRN	Specific Contribution
Jashan Chaudhary	25030422054	NLP pipeline implementation, model training/evaluation, project coordination.
Sanjeev Katragadda	25030422059	Dataset preparation, subgroup performance audit, results interpretation.
Agustya Gupta	25030422011	Explainability analysis, model-card content, limitations and risk analysis.
Ansh Sahani	25030422016	Report documentation, presentation preparation, final project organization.

Table 8. Team contribution breakdown.

18. Annexure (if required)

Annexure A – One-page model card

The one-page model card is submitted as a separate artifact. It summarizes the model, dataset, overall performance, two subgroup audits, explainability findings, limitations and responsible-use recommendations.

Annexure B – Key explainability output

Feature	Coefficient	Interpretation
ass	+1.4239	Strong hate-leaning feature
kill	+1.1557	Hate-leaning feature
gays	+1.1343	Identity-related hate-leaning feature
fuck	+1.1036	Profanity-related hate-leaning feature
stupid	+1.0709	Negative/offensive lexical signal
love	-1.1532	Non-hate-leaning feature
can	-1.0982	Non-hate-leaning feature
me	-1.0592	Non-hate-leaning feature
the	-1.0219	Non-hate-leaning feature
am	-0.9393	Non-hate-leaning feature

Table 9. Example top Logistic Regression coefficients used for explainability.

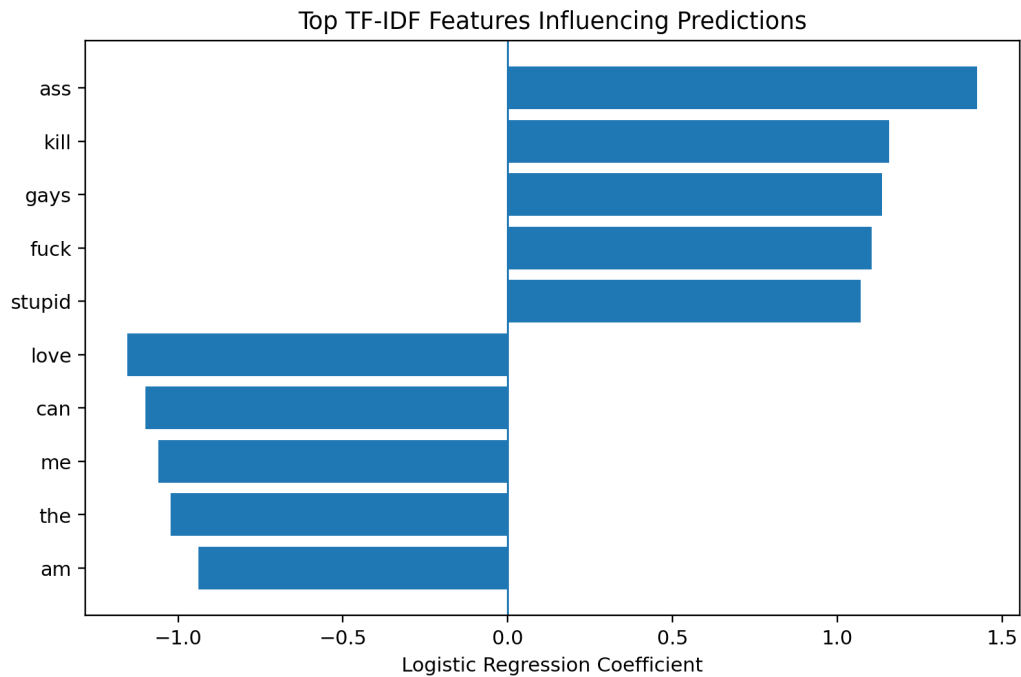


Figure 5. Top TF-IDF features influencing hate/non-hate predictions.